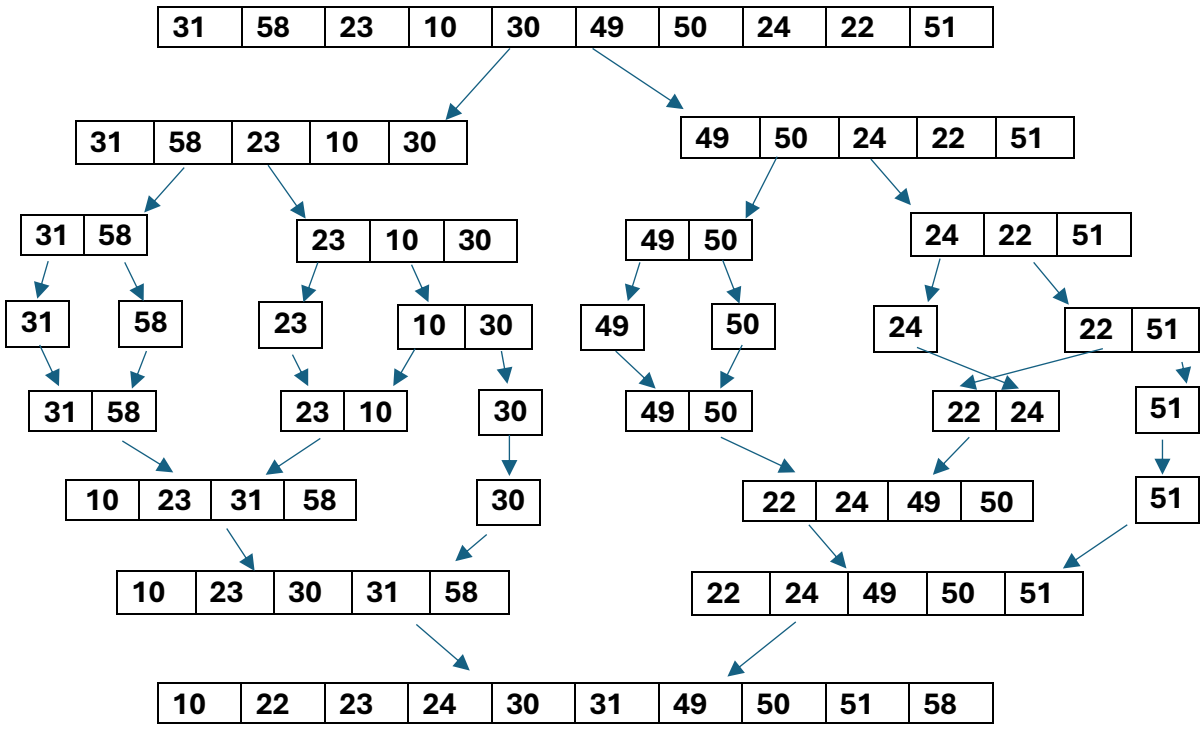
WEEK – 5

Quick Sort, Merge Sort and Binary Search Tree

MERGE SORT:

Given array: [31, 58, 23, 10, 30, 49, 50, 24, 22, 51]



CODE :

```
#include <stdio.h>

void merge(int arr[], int left, int mid, int right) {
  int i, j, k;
  int n1 = mid - left + 1;
  int n2 = right - mid;
  int L[n1], R[n2];
  for (i = 0; i < n1; i++)
    L[i] = arr[left + i];
  for (j = 0; j < n2; j++)
    R[j] = arr[mid + 1 + j];
  i = 0;
  j = 0;
```

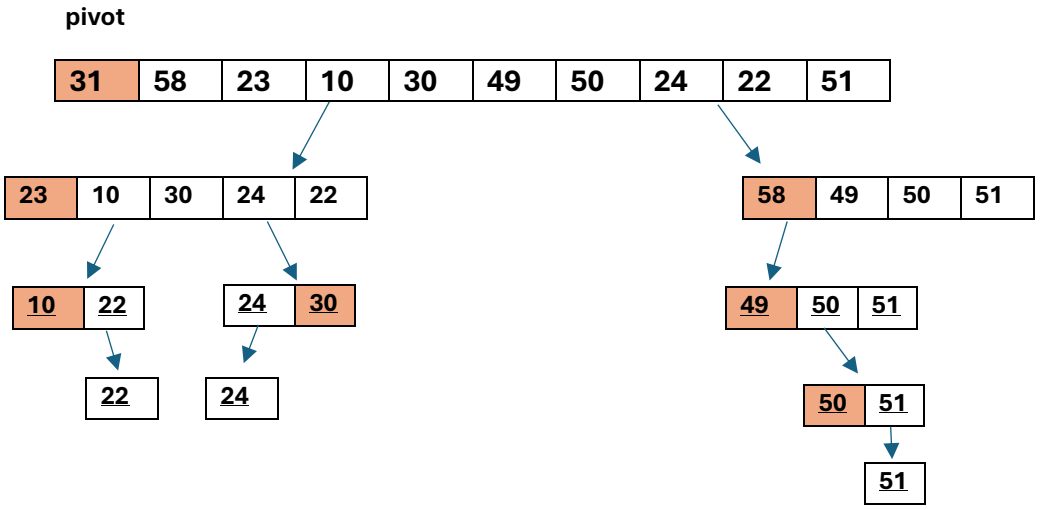
```
    k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
int main() {
    int arr[] = {31, 58, 23, 10, 30, 49, 50, 24, 22, 51};
    int n = sizeof(arr) / sizeof(arr[0]);

    mergeSort(arr, 0, n - 1);

    printf("Sorted array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

OUTPUT:

```
Sorted array:
10 22 23 24 30 31 49 50 51 58
```

QUICK SORT:

Given array: [31, 58, 23, 10, 30, 49, 50, 24, 22, 51]



CODE:

```
#include <stdio.h>
int partition(int a[], int low, int high) {
  int pivot = a[low];
  int i = low + 1;
  int j = high;
  int temp;

  while (i <= j) {
    while (i <= high && a[i] <= pivot)
      i++;
    while (a[j] > pivot)
      j--;

    if (i < j) {
      temp = a[i];
      a[i] = a[j];
      a[j] = temp;
    }
```

```
  temp = a[low];
  a[low] = a[j];
  a[j] = temp;

  return j;
}
void quickSort(int a[], int low, int high) {
  if (low < high) {
    int p = partition(a, low, high);

    quickSort(a, low, p - 1);
    quickSort(a, p + 1, high);
  }
}

int main() {
  int a[] = {31, 58, 23, 10, 30, 49, 50, 24, 22, 51};
  int n = sizeof(a) / sizeof(a[0]);
  quickSort(a, 0, n - 1);
  printf("Sorted array:\n");
  for (int i = 0; i < n; i++)
    printf("%d ", a[i]);
  printf("\n");
  return 0;
}
```

```
Sorted array:
10 22 23 24 30 31 49 50 51 58
```

Binary Search Tree:

```
      40
     /  \
   20    60
  /  \
10    30
```

Preorder: 40 → 20 → 10 → 30 → 60

Inorder: 10 → 20 → 30 → 40 → 60

Postorder: 10 → 30 → 20 → 60 → 40

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *left;
  struct Node *right;
};

struct Node* newNode(int value) {
  struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
  temp->data = value;
  temp->left = temp->right = NULL;
  return temp;
}

struct Node* insert(struct Node* root, int value) {
  if (root == NULL)
    return newNode(value);
  if (value < root->data)
    root->left = insert(root->left, value);
  else
    root->right = insert(root->right, value);
  return root;
}
void preorder(struct Node* root) {
  if (root != NULL) {
    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
  }
}
void inorder(struct Node* root) {
  if (root != NULL) {
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
  }
}
void postorder(struct Node* root) {
  if (root != NULL) {
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
```

```
    }
}

int main() {
    struct Node* root = NULL;
    int arr[] = {40, 20, 60, 10, 30};
    int n = 5;
    for (int i = 0; i < n; i++)
        root = insert(root, arr[i]);
    printf("Preorder Traversal: ");
    preorder(root);
    printf("\nInorder Traversal: ");
    inorder(root);
    printf("\nPostorder Traversal: ");
    postorder(root);
    printf("\n");
    return 0;
}
```

```
Preorder Traversal: 40 20 10 30 60
Inorder Traversal: 10 20 30 40 60
Postorder Traversal: 10 30 20 60 40
```